

# Guia: editar e adicionar rotinas na API (YottaDB)

Como funciona a API do moadir, como **editar** uma rotina existente e como **criar uma nova** e expô-la como endpoint.

## 1. Visão geral

A API (porta **9080**) é servida pelo **YDB Web Server** do YottaDB, rodando como o serviço systemd **ydbweb.service**. As rotinas MUMPS ficam em **/root/routines** na VPS.

Três peças formam um endpoint:

| Peça              | Onde                                                 | Papel                                       |
|-------------------|------------------------------------------------------|---------------------------------------------|
| Tabela de rotas   | /root/routines/_ydbweburl.m (label URLMAP)           | mapeia MÉTODO /caminho → label^ROTINA       |
| Rotina adaptadora | /root/routines/API*.m (ex.: APICLI, APIPRD)          | lê parâmetros, busca os dados, monta o JSON |
| Globais de dados  | ^FCL (clientes), ^EPR (produtos), ^DCP (prejuízo)... | os dados do ERP                             |

Dentro de uma rotina adaptadora, o web server fornece:

- `httpargs("nome")` → parâmetro da query string ( `?nome=...` )
- `httprsp(1)` → corpo da resposta
- `httprsp("mime")` → content-type (use `"text/plain; charset=utf-8"` )

A rotina `ALL.m` tem a **legenda** do que cada rotina do ERP faz (ex.: `E51` = Lista Produtos, `E14` = Edita Produtos, `FAT` = Faturamento). Útil para descobrir onde está a lógica/global de um assunto.

## Acesso à VPS

```
ssh Moadir          # = root@2.25.175.240
```

## 2. Regras de ouro (leia antes de mexer)

1. **Auto-relink está DESLIGADO.** Depois de alterar qualquer rotina você **tem que (a) recompilar e (b) reiniciar** o `ydbweb` — senão o servidor continua com a versão antiga em memória.
2. **É produção.** O `systemctl restart ydbweb` causa ~3 s de indisponibilidade da API (o dashboard dá erro nesse instante). Faça em horário tranquilo.
3. **Sempre faça backup** do que vai alterar ( `cp arquivo.m arquivo.m.bak` ), principalmente o `_ydbweburl.m` (um erro nele derruba **todos** os endpoints).
4. **Compile a rotina sozinha primeiro** para pegar erro de sintaxe **antes** de reiniciar o servidor.

## Comandos base (sempre carregue o ambiente primeiro)

```
source /root/ydb-api-env.sh      # define $ydb_dist e variáveis do YottaDB
cd /root/routines

$ydb_dist/mumps MINHAROT.m      # compila .m -> .o (mostra erros de sintaxe)
systemctl restart ydbweb.service # recarrega rotas e rotinas
journalctl -u ydbweb -n 50      # ver erros do servidor
```

## 3. Editar uma rotina existente

Exemplo: ajustar a `APIPRD` (produtos).

```
ssh Moadir
source /root/ydb-api-env.sh
cd /root/routines

cp APIPRD.m APIPRD.m.bak          # 1. backup
nano APIPRD.m                     # 2. edite

$ydb_dist/mumps APIPRD.m          # 3. compile (corrija erros até compilar limpo)
systemctl restart ydbweb.service  # 4. reinicie

curl -s "http://127.0.0.1:9080/api/produtos?limite=2" # 5. teste
```

Se algo der errado, restaure: `cp APIPRD.m.bak APIPRD.m && $ydb_dist/mumps APIPRD.m && systemctl restart ydbweb.service`.

---

## 4. Adicionar uma rotina nova e expor na API

---

Exemplo fictício: endpoint `GET /api/fornecedores` lendo um global `^F0R`.

### Passo 1 — (se necessário) colocar/adaptar a rotina de lógica

Se a lógica vier de outra rotina, copie o `.m` para `/root/routines` e ajuste para **MUMPS padrão do YottaDB**:

- Nada de ObjectScript/Caché (`##class(...)`, `$zconvert`, `&sql`) nem comandos de GUI proprietária (`ZGW`, `$ZF ...`) — o YottaDB não suporta.
- Confirme que as **globais** que ela lê (`^XXX`) existem neste banco.
- Compile só para checar sintaxe: `$ydb_dist/mumps MINHAROT.m`.

### Passo 2 — criar a rotina adaptadora `API*.m`

Espelhe a `APICLI` / `APIPRD`. Modelo:

```

APIFOR ; API REST – Fornecedores (read-only do ^F0R)
;
list ; GET /api/fornecedores?nome=<filtro>&limite=<n>
    NEW filtro,limite,json
    SET filtro=$$UPPER($GET(httpargs("nome")))
    SET limite=+$GET(httpargs("limite"))
    DO BUILDJSON(.json,filtro,limite)
    SET httprsp(1)=json
    SET httprsp("mime")="text/plain; charset=utf-8"
    QUIT
;
BUILDJSON(json,filtro,limite)
    NEW id,rec,nome,items,first,n,done
    SET items="",first=1,n=0,done=0,id=""
    FOR SET id=$ORDER(^F0R(id)) QUIT:id=""!done DO
        . SET rec=$GET(^F0R(id))
        . SET nome=$PIECE(rec,"^",1)
        . QUIT:nome=""
        . QUIT:(filtro=""&($FIND($$UPPER(nome),filtro)=0))
        . IF 'first SET items=items_"",
        . SET first=0
        . SET items=items_"{"id":"","_+id
        . SET items=items_"","nome":"","_$$CLEAN(nome)_"'"
        . SET items=items_"","cidade":"","_$$CLEAN($PIECE(rec,"^",4))_"'"
        . SET n=n+1
        . IF limite>0,n'<limite SET done=1
    SET json="{\"total\":\"_n\",\"fornecedores\":[\"_items\"]}"
    QUIT
;
CLEAN(s) QUIT $TRANSLATE(s,"\"_\"_$_CHAR(13)_$_CHAR(10))
UPPER(s) QUIT $TRANSLATE(s,"abcdefghijklmnopqrstuvwxyz","ABCDEFGHIJKLMNOPQRSTUVWXYZ")

```

### Passo 3 — registrar a rota em `_ydbweburl.m`

Adicione uma linha no URLMAP, **antes** do `;;zzzzz`:

```
;;GET /api/fornecedores list^APIFOR
```

(O arquivo já tem as rotas de clientes, produtos, relatório, etc.)

### Passo 4 — compilar e reiniciar

|                                                     |                                   |
|-----------------------------------------------------|-----------------------------------|
| <code>cp _ydbweburl.m _ydbweburl.m.bak</code>       | # backup do roteador!             |
| <code>\$ydb_dist/mumps APIFOR.m</code>              | # 1. compile a adaptadora sozinha |
| <code>\$ydb_dist/mumps APIFOR.m _ydbweburl.m</code> | # 2. compile as duas              |
| <code>systemctl restart ydbweb.service</code>       | # 3. recarrega rotas              |

### Passo 5 — testar

```

curl -s "http://127.0.0.1:9080/api/fornecedores?limite=5"
# e confira que os endpoints existentes não quebraram:
curl -s "http://127.0.0.1:9080/api/clientes?limite=1"

```

- **404** → a rota não entrou (revise o `_ydbweburl.m` e se reiniciou).
- **500 / corpo vazio** → erro na rotina (`journalctl -u ydbweb -n 50`).

## 5. Cuidados de MUMPS ao montar JSON

- **Sem precedência de operadores** — **é tudo da esquerda para a direita.** `a*1369+b*37` vira `(a*1369+b)*37`. Use parênteses: `(a*1369)+(b*37)`.
- **Sempre escape o texto** que entra no JSON com a função `CLEAN` (remove aspas, barra e quebras de linha). Strings cruas quebram o `JSON.parse` no app (o app React tem um sanitizador justamente por causa disso).
- **Concatenação** é com `_` (underscore). Aspas dentro de string `M` se escrevem duplicando: `""` vira uma aspa.
- **Números com decimais:** `$JUSTIFY(valor/100,0,2)` formata com 2 casas.
- **\$ORDER** percorre subscripts; **\$PIECE(rec,"^",n)** pega o campo `n` (campos separados por `^`).

⚠ A função `$$zh^c` usada por rotinas do ERP (ex.: para achar as casas decimais de uma unidade em `^EUN`) **não existe nesta VPS** (a rotina `c` não está instalada). O algoritmo do hash foi replicado dentro da `APIPRD` (labels `ZH / H`) — copie de lá se precisar.

## 6. Modelo de dados conhecido

| Assunto  | Global                                      | Rotina API                                                           | Observações                                                                                                                                                                                   |
|----------|---------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clientes | <code>^FCL(id\168, id\2, id_"1"/"2")</code> | <code>APICLI</code>                                                  | <code>/api/clientes</code> , <code>/api/cliente?id=</code>                                                                                                                                    |
| Produtos | <code>^EPR(k\336, k\4, k)</code>            | <code>APIPRD</code>                                                  | <code>/api/produtos</code> . Campos: 1=descrição, 2=unidade, 10=custo, 11=venda, 13=local, 18/19=estoque. <b>Preços ÷100; estoque</b> com casas decimais = <code>^EUN(hash da unidade)</code> |
| Prejuízo | <code>^DCP</code>                           | <code>APIRELPREJ</code><br><code>/</code><br><code>APIRELHTML</code> | <code>/api/relatorio?dini=&amp;dfim=</code>                                                                                                                                                   |

## 7. Mostrar no app (frontend)

Depois que o endpoint existir, o lado React é:

1. `app/lib/api.ts` — tipo + função (`listarX`) que faz `getJSON` da nova URL.
2. `app/routes/x.tsx` — página com `loader` chamando a função + `DataTable`.
3. `app/routes.ts` — registrar a rota.
4. `app/components/app-sidebar.tsx` — item de menu (opcional).
5. `git push` → a VPS publica sozinha em ~1 min (deploy pull via systemd timer).

Veja `docs/` e as páginas existentes (`clientes.tsx`, `produtos.tsx`) como modelo.

## 8. Checklist rápido

```
[ ] ssh Moadir && source /root/ydb-api-env.sh && cd /root/routines
[ ] backup das rotinas que vou mexer (cp X.m X.m.bak) — inclusive _ydbweburl.m
[ ] criar/editar a rotina API*.m (escapar JSON; cuidado com L→R do M)
[ ] registrar a rota em _ydbweburl.m (antes do ;zzzzz)
[ ] $ydb_dist/mumps APIX.m (compila a adaptadora sozinha)
[ ] $ydb_dist/mumps APIX.m _ydbweburl.m
[ ] systemctl restart ydbweb.service (em horário tranquilo)
[ ] curl o endpoint novo + um existente p/ garantir que nada quebrou
[ ] se quebrou: restaurar .bak, recompilar, reiniciar
```